

ASR Error Correction using Large Language Models

Rao Ma*, Mengjie Qian*, Mark Gales, *Fellow, IEEE*, Kate Knill, *Senior Member, IEEE*

Abstract—Error correction (EC) models play a crucial role in refining Automatic Speech Recognition (ASR) transcriptions, enhancing the readability and quality of transcriptions. Without requiring access to the underlying code or model weights, EC can improve performance and provide domain adaptation for black-box ASR systems. This work investigates the use of large language models (LLMs) for error correction across diverse scenarios. 1-best ASR hypotheses are commonly used as the input to EC models. We propose building high-performance EC models using ASR N-best lists which should provide more contextual information for the correction process. Additionally, the generation process of a standard EC model is unrestricted in the sense that any output sequence can be generated. For some scenarios, such as unseen domains, this flexibility may impact performance. To address this, we introduce a constrained decoding approach based on the N-best list or an ASR lattice. Finally, most EC models are trained for a specific ASR system requiring retraining whenever the underlying ASR system is changed. This paper explores the ability of EC models to operate on the output of different ASR systems. This concept is further extended to zero-shot error correction using LLMs, such as ChatGPT. Experiments on three standard datasets demonstrate the efficacy of our proposed methods for both Transducer and attention-based encoder-decoder ASR systems. In addition, the proposed method can serve as an effective method for model ensembling.

Index Terms—Automatic speech recognition, error correction, large language model, supervised training, zero-shot prompting

I. INTRODUCTION

Automatic speech recognition (ASR) aims to transcribe speech audio into text and is the key component for human-computer interaction [1]. In recent years, the performance of ASR technology has dramatically advanced, evolving from traditional Hidden Markov Model (HMM)-based architectures to modern end-to-end (E2E) systems like Listen, Attend and Spell (LAS) or RNN-T [2]–[5]. Large-scale models such as Whisper [6] and Google USM [7] have demonstrated state-of-the-art performance, leveraging vast amounts of labeled and unlabeled speech data, which can be costly to obtain. While achieving impressive results on test sets, practical deployment of ASR systems encounters challenges, especially when faced with domain-specific or previously unseen speech data.

Accessing ASR services via APIs has emerged as a popular alternative to training in-house models, offering a pragmatic and economical choice. Fine-tuning these models for specific tasks, however, is often impractical due to restricted access to proprietary models. Various approaches have been proposed

to enhance such restricted access ASR systems. Two common methods are language model (LM) rescoring and error correction (EC). LM rescoring involves reranking the N-best list generated by the ASR system using an external LM, which can improve the overall performance of the ASR system [8]. Recent research, however, has shown that E2E ASR models often learn an internal language model (ILM) on the training data, which can reduce the effectiveness of traditional shallow fusion techniques [9]. Methods to address the impact of ILMs, such as those proposed by [10]–[12], generally involve code modification during the inference stage, therefore they are out of the scope of discussion in this paper.

Error correction, applied as a post-processing step for ASR systems, offers a promising alternative [13]–[15]. This approach requires only the decoding hypotheses and reference data to train a model, eliminating the need for deep access to the ASR system. Early work in this area focused on rule-based systems which rely on statistical analysis [16]. More recent developments have introduced end-to-end models with attention modules, which can automatically identify errors within sentences and learn to generate the correct counterparts implicitly [17]–[19]. Large-scale pre-trained language models (PLMs) are trained on massive and diverse text datasets, far exceeding the scale of data used in ASR training. Approaches to transfer knowledge from PLMs for accurate ASR error detection and correction have been recently proposed. For example, Hrinchuk et al. [14] propose a Transformer-based architecture to “translate” an ASR model output into grammatically and semantically correct text. Zhao et al. [15] introduce a BART-based semantic correction system for the Mandarin ASR system. Shen et al. [20] propose a masking strategy to train the model to correct the original error tokens and predict the masked tokens based on their context information. Ma et al. [21], [22] propose an N-best T5 model based on pre-trained T5 models to perform error correction using the ASR N-best list. By fine-tuning these pre-trained large language models (LLMs), the implicit knowledge acquired from vast amounts of text data can be effectively transferred to the target error correction task.

The recent advent of generative LLMs has further advanced EC techniques. Within the field of NLP, studies such as [23], [24] have applied ChatGPT models to grammatical error correction tasks. In the context of ASR error correction, previous research has examined zero-shot performance using LLMs [25]. Everson et al. [26] utilized word confusion networks generated by the ASR system and performed EC with

* Equal Contribution.

in-context learning, demonstrating improved performance with one-shot examples compared to 1-best hypotheses. Chen et al. [27] generated N-best lists in the ASR decoding and built LLM EC systems using various methods including fine-tuning, LoRA tuning, and in-context learning. Hu et al. developed a multi-modal EC model incorporating audio as an additional input [28] and used a cloze-test task approach instead of a generative correction method. Additionally, Li et al. [29] explored knowledge transfer within LLMs by fine-tuning a multilingual LLM across various languages to correct 1-best hypothesis errors from different speech foundation models.

Previous research has also explored various methods to improve ASR error correction by leveraging N-best lists, which offer richer information compared to single 1-best hypotheses. For instance, Guo et al. [18] generates an 8-best list with the ASR model and rescored candidates with an LSTM language model [30]. Zhu et al. [31] concatenated N-best hypotheses for input to a bidirectional encoder, and Leng et al. [32] investigated non-autoregressive models with similar approaches. More recent work by Ma et al. [25] and Chen et al. [27] has integrated N-best lists with generative LLMs to enhance error correction performance.

Building on these advances, our paper introduces a novel approach that uses LLMs to improve ASR error correction. We compare fine-tuning versus zero-shot error correction methods and investigate how ASR N-best lists can be effectively utilized. A major contribution of our work is the innovative use of ASR N-best lists as extended inputs, which provides richer context and more accurate cues for error correction. Additionally, we introduce advanced decoding strategies, including constrained decoding, to enhance model robustness and alignment with the original utterance. Our approach also addresses data contamination concerns by developing methods to evaluate the impact of training data biases. By integrating these elements, our study sets a new benchmark in ASR error correction, advancing the state-of-the-art in the field.

This paper is structured as follows. In Section II we present the error correction method utilizing foundation language models, including a supervised approach employing the T5 model and a zero-shot approach based on generative LLMs. Section III introduces several decoding algorithms to build more robust ASR error correction systems, aiming to address the inherent problems of the standard beam search approach. In Section IV, we describe the method used to investigate data contamination. The experimental setup and results are detailed in Section V, while Section VI covers N-best analysis, an ablation study of the proposed approach, and a discussion on data contamination. Finally, Section VII presents the conclusions.

II. ASR ERROR CORRECTION MODELS

A. ASR Error Correction using N-best lists

Building upon previous findings, we explore new methodologies for effectively incorporating supplementary ASR outputs into LLM-based EC models. One promising approach involves utilizing N-best ASR hypotheses, which are generated by the ASR system as a byproduct of the beam search process [33], [34]. The integration of N-best T5, using the

N-best list as input, has demonstrated significant performance improvements in error correction compared to the original model [21]. The rationale behind this is that the N-best list contains alternative sequences that have a strong possibility of being the correct transcription, thus providing valuable cues for the EC model during predictions. We modify the input of the EC model to be sentences in the N-best list, sorted based on ASR scores and concatenated with a special token, improving both interpretability and effectiveness.

In our study, we introduce methods for developing both fine-tuning and zero-shot error correction models using ASR N-best lists and propose several decoding methods. In traditional ASR error correction models, the decoding process allows the model to generate any sequence based on the given context, meaning that the output space is not constrained, which is denoted as unconstrained decoding (**uncon**). When an N-best list is used, we can constrain the model to allow it to generate from a limited space. For this, we propose N-best constrained decoding (**constr**) and N-best closest (**closest**). The details of these decoding methods will be introduced in Section III.

B. Fine-tuning based Approach

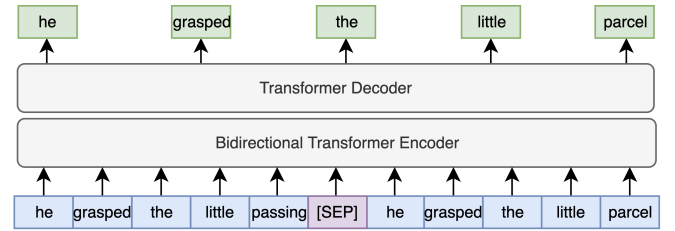


Fig. 1. The model structure of a supervised error correction model using ASR N-best lists as input. Here, we set N to 2 for illustration.

EC models aim to correct recognition errors in ASR transcriptions, serving as a post-processing step for speech recognition. A standard supervised EC model adopts an E2E structure, taking the ASR transcription as input, and is trained to generate the corrected sentence [14], [15], [18]. Adapting an EC model from a PLM yields superior performance when compared to training the EC model from scratch, as this approach leverages the prior knowledge embedded in the language models [21]. When training an EC model, direct access to the ASR system is unnecessary as only the decoded hypotheses are required. This flexibility in data accessibility makes the method highly practical, especially in situations where adapting a black-box, cloud-based speech-to-text system is essential.

The structure of our proposed EC model is illustrated in Figure 1, where ASR N-best lists are given as input to the encoder and the model is trained to generate the manual reference. Here, sentences are concatenated with a special token [SEP]. The model is trained to automatically detect errors and generate the corrected hypothesis on specific training data. For supervised models, generalization capability is crucial, as it enhances their applicability across various practical scenarios. A model that generalizes well can be used in diverse and practical contexts without the need for further updates. In

zero-shot uncon

Input: Perform error correction on the top3 outputs generated by an Automatic Speech Recognition (ASR) system. The ASR hypotheses, listed in order of their ASR posterior score, are as follows:

<hypothesis1> he grasped the little passing </hypothesis1>
 <hypothesis2> he grasped the little parcel </hypothesis2>
 <hypothesis3> he grasped the little puzzle </hypothesis3>

Please provide the corrected top1 ASR transcription of the given utterance only, do not add any explanations or other words.

ChatGPT: he grasped the little parcel

zero-shot constr

Input: Perform language model rescoring based on the top3 outputs generated by an Automatic Speech Recognition (ASR) system. The ASR hypotheses, listed in order of their ASR posterior score, are as follows:

<option1> he grasped the little passing </option1>
 <option2> he grasped the little parcel </option2>
 <option3> he grasped the little puzzle </option3>

Please output the selected top1 ASR transcription as <option?> The selected top1 ASR transcription </option?>.

ChatGPT: The selected top1 ASR transcription is <option2> he grasped the little parcel </option2>.

Fig. 2. Prompt design for zero-shot ASR error correction. Here we use a 3-best list generated by the ASR system as input to ChatGPT for illustration.

our experiments, we applied a model trained on transcriptions of corpus A generated by a specific ASR system to out-of-domain test sets and outputs from other ASR systems. By doing this, we demonstrate the generalization ability of our proposed method, highlighting its robustness and adaptability across varying ASR outputs and domains.

C. Zero-shot Approach

Supervised training has long been popular for developing EC systems, however, it requires the availability of training data and the systems can be computationally expensive to build. To address these constraints, we present our approach to utilizing generative LLMs for zero-shot error correction, using ChatGPT as an illustrative example. This task can be challenging as ChatGPT lacks prior knowledge about the error patterns of the ASR system and has no access to the original utterance. To mitigate the difficulty, similar to the supervised approach, we provide hypotheses from the N-best list as valuable hints, helping the model to detect and correct errors effectively. The prompts we used in the experiments are shown in Figure 2. In the prompt design, hypotheses are sorted by the descending ASR posterior score. Additionally, tags such as <hypothesis1> and </hypothesis1> enclose each N-best hypothesis. We experimented with other input formats, such as using numbers instead of tags or employing plain sentences without explicitly specified order, but these variants showed degraded performance compared to our chosen prompt. In the ablation study detailed in Section VI-A, we highlight the importance of using a reasonable number of N for the model to achieve optimal performance. When only the top one ASR hypothesis is used as input, ChatGPT-based error correction may experience a degradation in performance. Additionally, initial experiments explored the few-shot setting, revealing unstable performance improvements compared to the zero-shot approach and higher computational costs. Therefore we mainly focus on the zero-shot results in this paper.

III. ASR ERROR CORRECTION DECODING

Previous sections introduced the proposed fine-tuning and zero-shot error correction methods. Specifically, we highlighted how to incorporate more information into the input space of the proposed model with ASR N-best lists and

evaluate the impact on performance. Another intriguing aspect is guiding the decoding process to achieve controllable generation. In this section, we delve into this challenge, exploring methods to direct the model’s output in a more controlled and predictable manner.

A. Unconstrained Decoding

For an ASR error correction model with parameters θ_{EC} , the N-best input is denoted as $\mathcal{Z} = \{\hat{z}^{(1)}, \hat{z}^{(2)}, \dots, \hat{z}^{(n)}\}$. Our decoding objective is to find $\hat{y}_{uncon}$ that satisfies

$$\hat{y}_{uncon} = \arg \max_y \log P(y|\mathcal{Z}; \theta_{EC}) \quad (1)$$

where y presents potential output sequences. Given the computational cost of finding the globally optimal sequence, heuristic algorithms such as beam search are commonly used for decoding. In this context, the decoding method is referred to as unconstrained decoding (**uncon**), as no explicit constraints are applied to the generated sequences.

Beam search is a practical tool for approximating optimal generation results across the entire decoding space, balancing efficiency and performance. However, this method grants the model too much freedom, limiting our control over the decoding process [35]. Specifically, we aim for the proposed model to retain correct words from the original transcription and only correct inaccurate ones. In addition, the model is expected to generate homophones for detected erroneous words. While these aspects can be implicitly learned from the training data, there is no guarantee they will be applied during decoding. The model might produce synonyms with high embedding similarity to words in the reference text, which can be problematic. To address these concerns, we introduce several alternative decoding algorithms to enhance the model’s ability to achieve the desired decoding objectives. These methods aim to exert more control over the output, ensuring that corrections are precise and that the generated sequences meet specific criteria for accuracy and relevance.

B. N-best Constrained Decoding

In unconstrained decoding, the decoding space of the EC model is unbounded. However, we want the generated correction results to closely resemble the original utterance. One

method to introduce constraints on the decoding space involves leveraging the ASR N-best list, which comprises the top N hypotheses generated by the ASR system, representing the transcriptions most likely to be correct given the input audio. This approach, denoted as N-best constrained decoding (**constr**), forces the model to only generate sentences within the ASR N-best list.

Furthermore, each path in the N-best list is associated with a score calculated by the ASR system, indicating the likelihood of it being the correct output. These scores can be combined with the scores from the EC model, using an interpolation weight λ to gain insights from both models. To be more specific, the decoding result $\hat{y}_{\text{constr}}$ is derived by maximizing the equation:

$$\hat{y}_{\text{constr}} = \arg \max_{y \in \mathcal{Z}} [(1 - \lambda) \cdot \log P(y|x; \theta_{\text{ASR}}) + \lambda \cdot \log P(y|\mathcal{Z}; \theta_{\text{EC}})] \quad (2)$$

where x and $\mathcal{Z}$ denote the input acoustic features of the ASR system and the obtained ASR N-best list, respectively. When λ is set to 1, the scores from the ASR system are ignored, and only the probabilities from the EC model are considered. This approach requires obtaining the probability scores from the EC model, which can be implemented in the supervised EC method. Section V-B will demonstrate its effectiveness and highlight situations where its utility becomes evident.

In zero-shot EC scenarios, the method is applied differently. Instead of generating a correction from scratch, ChatGPT is tasked with selecting the most likely correct ASR transcription from a list of candidates. As illustrated in Figure 2, all the N-best sentences are listed as input, such as `<option1>` ASR hypothesis `</option1>`, and ChatGPT is instructed to return the selected option in the format of `<option?>`. The selected ASR transcription `</option?>`. While this method is similar to language model rescoring to some extent, it differs in that the selection occurs in a single step. More importantly, ChatGPT sees all the candidates before determining the best one. This contrasts with the rescoring process, where language model scores are generated individually for each of the N-best hypotheses without considering their similarity and correlation.

C. N-best Closest Decoding

The closest mapping method (**closest**) is based on the assumption that during unconstrained error correction, LLMs first choose the best hypothesis from the given N-best list and then modify this sentence to yield the final output. In experiments, we aim to identify this “closest match” by performing a reverse process, in which we locate the hypothesis within the ASR N-best list that has the smallest Levenshtein distance to the unconstrained generation result, as shown in Equation 3:

$$\begin{aligned} \hat{y}_{\text{uncon}} &= \arg \max_y \log P(y|\mathcal{Z}; \theta_{\text{EC}}) \\ \hat{y}_{\text{closest}} &= \arg \min_{z \in \mathcal{Z}} \text{LevenshteinDist}(\hat{y}_{\text{uncon}}, z) \end{aligned} \quad (3)$$

To illustrate, consider the zero-shot uncon example in Figure 2, where the Levenshtein distance of the ChatGPT output to the 3-best ASR hypotheses is 1, 0, 1, respectively. In this scenario, the second hypothesis would be selected as the corrected

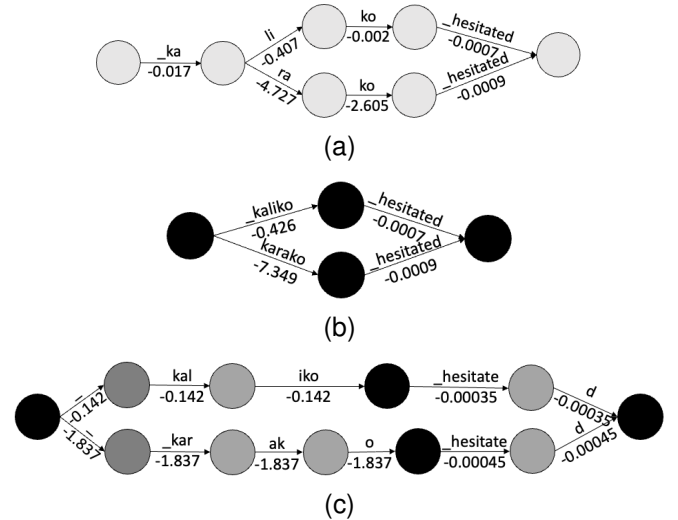


Fig. 3. (a) Example BPE-level lattice generated in ASR decoding. (b) Converted word lattice. (c) Converted lattice with LLM BPE tokens.

result for the given utterance. Notably, this method is different from the N-best constrained method, which explicitly tasks the LLMs with selecting from the N-best list. The N-best closest approach does not constrain the output space initially but rather finds the closest match within the N-best list after the unconstrained generation.

D. Lattice Constrained Decoding

Algorithm 1 Lattice Constrained Decoding for N-best T5

Data: lattice node set $\mathcal{V}$, lattice edge set $\mathcal{E}$, beam width b , T5 encoder outputs $\{\mathbf{h}_j\}$

```

1:  $Q \leftarrow \text{topological\_sort}(\mathcal{V})$ 
2: for  $v$  in  $\mathcal{V}$  do
3:    $H_v \leftarrow \text{min\_heap}()$ 
4: end for
5:  $n_0.\text{history} = \epsilon, n_0.\text{score} = 0$ 
6:  $H_{\text{start}}.\text{put}(n_0)$ 
7: for  $v$  in  $Q$  do
8:    $\mathbf{o} = \text{Decoder}(\{\mathbf{h}_j\}, n.\text{history}, v.\text{word})$ 
9:   for  $\langle v, x \rangle$  in  $\mathcal{E}$  do
10:    for  $n$  in  $H_v$  do
11:       $n'.\text{history} = \text{concat}(n.\text{history}, v.\text{word})$ 
12:       $n'.\text{score} = n.\text{score} + \lambda \cdot \log(\mathbf{o}[x.\text{word}]) + (1 - \lambda) \cdot \log s_{v,x}$ 
13:      if  $H_x.\text{size} \geq b \wedge H_x.\text{score}.\text{min}() < n'.\text{score}$  then
14:         $H_x.\text{remove\_min}()$ 
15:      end if
16:      if  $H_x.\text{size} < b$  then
17:         $H_x.\text{put}(n')$ 
18:      end if
19:    end for
20:  end for
21: end for
22:  $I = \text{max\_heap}(H_{\text{end}}.\text{items})$ 
23: return  $I.\text{max}()$ 

```

While an ASR N-best list is useful for capturing likely correct candidates, it represents only a limited set of possible decoding outputs. Instead of strictly constraining to the N-best list, we can explore a more flexible approach by expanding the decoding space to include the lattice generated through path merging. As depicted in Equation 4, we focus on the paths $\mathcal{G}$ within the lattice and integrate ASR scores during decoding. This lattice-constrained decoding approach enhances

Your task is to create a four-choice quiz by replacing the words in the provided "Input Text" with their contextually relevant synonyms. The meaning and sentence structure of the four options must exactly match every detail in the Input Text. You must not include the provided Input Text as an option.

- (1) You generate distinct options based on the provided Input Text;
- (2) The only difference between options is word-level perturbations;
- (3) Options are ordered;
- (4) There is not any extra explanation;
- (5) You follow the following "Format" to generate options.
- (6) Don't add punctuation and use all lowercase as the input text.

--

Input Text:
surrounded by sounds of church bell in sunlight and birdsong

--

Format:
A)
B)
C)
D)

Fig. 4. Prompt for generating options for the data contamination quiz.

flexibility, allowing for a broader exploration of the potential corrections beyond the N-best list, and hence has the potential to improve overall decoding accuracy.

$$\hat{y}_{\text{lattice}} = \arg \max_{y \in \mathcal{G}} [(1 - \lambda) \cdot \log P(y|x; \theta_{\text{ASR}}) + \lambda \cdot \log P(y|\mathcal{Z}; \theta_{\text{EC}})] \quad (4)$$

Since this method requires ASR decoding probabilities at each time step to generate the lattice, it needs access to the ASR model, hence it is only applicable in certain scenarios. We tested this approach in the supervised EC model, N-best T5 model to be specific. Notably, the ASR model and the pre-trained language model employ different tokenizers. Consequently, we need to convert the original lattice into an equivalent form suitable for the N-best T5 to process. To achieve this, the lattice with ASR BPE tokens is first converted into a word lattice with dynamic programming, as shown in Figure 3b. Then the words along the edges are segmented into BPE tokens using the T5 tokenizer, as demonstrated in Figure 3c. The decoding algorithm is adapted from [36], and the details presented in Algorithm 1. This method requires access to the beam search candidates in the decoding process, we only applied it to T5 rather than the closed-form LLM ChatGPT in the experiments. Our preliminary experiments indicate that performance remains consistent as beam sizes increase while the associated costs rise. Therefore in our experiments, we run the lattice-constrained decoding with the beam size of 1.

IV. DATA CONTAMINATION

In the context of commercial use, most generative LLMs are developed with closed access to their training data. This lack of transparency presents challenges in determining if specific test data has been encountered during the model's pre-training phase. Utilizing test data that has already been exposed to the LLM during pre-training can skew evaluations, resulting in inaccurate assessments of model performance – an issue commonly referred to as data contamination [37], [38].

To address this concern, we adapted the method from [39] to measure the degree of the data contamination problem on the datasets we utilized in this paper. For each test utterance, we use GPT-4 to rewrite the original manual reference and

Instructions: You are provided with a three-choice quiz. Your task is to correctly select the option that exactly corresponds to an instance from the test split of the MGB-3 dataset. When selecting the option, you must ensure that you follow the following rules:

- (1) You must ensure that you only generate a single option letter as your answer.
- (2) If you do not know the dataset of the correct answer, you must select option "C) None of the provided options."

Hint: While all the following options seem similar in terms of meaning, there is only one option that reflects the exact wording with respect to the original sentence.

--

A) surrounded by sounds of church bell in sunlight and birdsong
B) encircled by noises of chapel chime in sunshine and avian melodies
C) None of the provided options.

Fig. 5. Demonstration of the data contamination quiz. Here, answer A) is selected from the original test set, and B) is the paraphrased one. We also switch the order of options A) and B) to mitigate the possible positional bias.

TABLE I
STATISTICS OF ASR TEST SETS USED IN THE EXPERIMENTS.

Dataset	Subset	# Utts	# Words	Hours
LibriSpeech	train	281,231	9.4M	960.9
	test_clean	2,620	53K	5.4
	test_other	2,939	52K	5.1
TED-LIUM3	test	1,155	28K	2.6
Artie Bias	test	1,712	15K	2.4

generate paraphrased candidates. As presented in Figure 4, instructions are given to GPT-4 to generate options by altering the words without affecting the sentence's meaning. We then randomly select one paraphrased sentence and feed it to LLM along with the original test sample to evaluate the degree of data contamination.

Figure 5 depicts the basic format of the designed 3-choice data contamination quiz. In this example, both sentences convey similar meanings while answer A) is copied from the test set and answer B) is the paraphrased one generated by a LLM. In addition, we provide option C), which denotes the non-appearance of both sentences. If the model generates A) in this scenario, it will suggest potential data contamination in the model pre-training. We estimate the level of contamination with the percentage of the test samples where the model selected the original sentence. This value is expected to be close to zero for a model that has never been pre-trained on the test set. Previous works have observed implicit positional bias in the LLM generation process [40]. To target this problem, we run the method twice, changing the order of the given options, and compute the overall classification performance.

V. EXPERIMENTS

A. Experimental Setup

Three standard datasets are used for training and evaluation, namely LibriSpeech [41], TED-LIUM3 [42], and Artie bias corpus [43]. LibriSpeech is an audiobook-based English speech corpus, which covers a wide range of speakers with different accents. TED-LIUM3 is an audio dataset collected from TED talks, encompassing various topics such as science, education, and entertainment. The Artie bias corpus is a subset of the Common Voice dataset [44] which is also read speech. Detailed statistics of these datasets are listed in Table I.

The experiments were conducted on two ASR models: a Conformer-Transducer model [45] and the OpenAI Whisper ASR [6]. The Conformer-Transducer’s encoder features 12 Conformer layers with a hidden size of 512 and its predictor has one LSTM layer. Both the jointer and predictor have hidden dimensions of 512. This model is trained on the 960hr LibriSpeech dataset following the ESPnet recipe [46]. SpecAugment [47] and speed perturbation are used for training data augmentation. We use a beam size of 10 in the decoding and save the generated top hypotheses. For Whisper, we adopt the small.en model due to its comparative performance to larger models and faster processing speed. The original decoding result only returns the 1-best hypothesis and the sentence-level confidence score for each utterance. We modified the code to also save the 10-best lists during inference and to extract token-level softmax probabilities for calculating word-level ASR confidence scores. This is to simulate the scenario where the ASR service provides extra information for downstream tasks. In the evaluation, we run text normalization scripts on both reference and hypothesis before calculating WER results following [6].

For our fine-tuning ASR error correction method, we experimented with a T5 base model, an encoder-decoder model pre-trained on various text-to-text tasks. We aimed to build each EC model using the same ASR training corpus with input transcriptions generated by the corresponding trained ASR system. The Transducer model fit the ASR training set so well that it achieved an extremely low WER, making the development of an ASR error correction model impractical. To address this, we employed data augmentation methods to generate erroneous transcriptions for training the correction model. Specifically, we applied SpecAugment to each utterance in the ASR decoding process. We utilized two frequency masks with $F = 30$, eight time masks with $T = 40$, and time warping with $W = 40$ on the training speech data. In the decoding results, sentences with WERs higher than 0.25 were filtered out, resulting in a training text corpus comprising 262K sentence pairs. The T5 model is fine-tuned for 3 epochs on this corpus using the AdamW [48] optimizer. The initial learning rate is set to $5e-5$ and the training batch size is 32. A dropout rate of 0.1 is applied to the network to prevent overfitting. For the zero-shot experiments, we used two versions of ChatGPT models: gpt-3.5-turbo-0613 and gpt-4-0125-preview, which are abbreviated into GPT-3.5 and GPT-4 in the paper.

B. Experiments on Fine-tuning Approach

Table II presents the results of two supervised error correction models trained for two ASR models with their outputs, evaluated on the LibriSpeech test_clean and test_other datasets. For the Transducer model, the oracle WER improves by 33.5% with the 5-best list and 38.4% with the 10-best list on the test_other set compared to the baseline. Similarly, for the Whisper model, the 5-best and 10-best lists achieve oracle WER improvements of 29.0% and 35.8% on the same set. These results suggest that the N-best lists have potential in helping the models recover the correct transcription.

We compare the WER results of the 10-best T5 model using various decoding algorithms, as detailed in Section III. The EC

TABLE II
RESULTS (% WER) FOR A CONFORMER-TRANSDUCER SYSTEM AND WHISPER USING A T5 ERROR CORRECTION MODEL, COMPARING DIFFERENT (UN)CONSTRAINED DECODING ALGORITHMS.

System		Transducer		Whisper	
		clean	other	clean	other
Baseline		2.79	6.90	3.52	7.37
5-best Oracle		1.42	4.59	2.38	5.24
10-best Oracle		1.31	4.25	2.14	4.73
10-best T5	uncon	2.54	6.37	2.90	6.39
	constr	2.42	6.15	3.10	6.69
	closest	2.50	6.24	3.11	6.52
	lattice	2.41	6.10	-	-

model trained for the Transducer model with its outputs shows improved performance when additional constraints are applied during decoding. Specifically, in the constrained decoding process, optimal interpolation weight λ is searched within the range [0.0, 1.0] with a grid size of 0.05. With N-best constrained decoding and closest mapping, the model effectively generates homophones for the mistaken words. Lattice-constrained decoding, which provides more potential paths than an N-best list and thus has a lower oracle WER, results in slightly better performance on the test sets (13.2% and 11.6% WERR on test_clean and test_other sets, respectively). Unlike the N-best T5 for the Transducer ASR, the EC model tailored for Whisper outputs adeptly detects and corrects errors for LibriSpeech corpus utterances in unconstrained decoding, yielding WERR of 17.6% on test_clean and 13.3% on test_other. This indicates that the model has effectively learned to correct errors in the Whisper ASR from its training on 960hr LibriSpeech speech. Constrained decoding yields less improvement. This limits the model to the N-best list which might have restricted performance compared to unconstrained decoding. Given the strong results from unconstrained decoding with Whisper and the complexity involved in generating lattices, lattice-constrained decoding is not considered here.

Results in Table II have shown a significant improvement in recognition accuracy using the fine-tuned N-best T5 method on the target test sets. However, it will be more applicable if it can be applied to out-of-domain datasets or on outputs from a different ASR system. We therefore evaluate the generalization ability of the fine-tuning EC method from both perspectives.

Generalization on out-of-domain datasets: To examine the generalization ability of the proposed method on out-of-domain datasets, we directly applied the EC models trained on LibriSpeech transcriptions to the ASR outputs from other test sets (TED-LIUM and Artie bias corpus) without fine-tuning. The results are presented in Table III, and we studied the performance of two supervised EC models trained with Transducer ASR and Whisper ASR outputs, respectively. In the baseline results, Whisper achieved much lower WERs for these datasets than the Transducer model, which was only trained with LibriSpeech data. The N-best T5 model, trained for Transducer outputs, improves ASR performance on out-of-domain datasets like TED and Artie without any fine-tuning, in both unconstrained and constrained decoding settings. Un-

TABLE III

ERROR CORRECTION RESULTS (% WER) FOR CONFORMER-TRANSDUCER AND WHISPER SYSTEMS ON OUT-OF-DOMAIN ASR TEST SETS AND ON OUTPUTS FROM A DIFFERENT ASR.

EC training source EC applied to source Test sets	Transducer		Whisper		Whisper	
	Transducer TED	Artie	Whisper TED	Whisper Artie	Transducer clean	Transducer other
Baseline	13.53	23.67	3.89	9.03	2.79	6.90
10-best Oracle	10.21	16.69	2.59	5.60	1.31	4.25
10-best T5	uncon	12.00	21.24	4.56	9.16	3.86
	constr	12.12	21.36	3.64	8.14	2.62
					6.65	

constrained decoding yields slightly better performance than N-best constrained decoding, with relative word-error-rates (WERRs) of 11.3% and 10.3% on TED and Artie, respectively. Although Whisper demonstrates very low baseline WERs, making further improvements challenging in a zero-shot transfer setting, the N-best T5 model still manages to reduce WER by 6.4% and 9.9% for TED and Artie, respectively, using N-best constrained decoding.

Generalization on other ASR systems: The practical utility of the EC model increases significantly if it can effectively correct outputs from ASR systems different from the one used for its training. In Table III, we investigated this aspect of the model’s generalization ability. We applied the EC model trained with LibriSpeech transcriptions generated with the Whisper model directly to the ASR outputs from the Transducer model. Under unconstrained decoding conditions, the model struggled to achieve performance gains, highlighting the challenge of domain mismatch. However, employing constrained decoding successfully improved performance across both LibriSpeech test_clean and test_other datasets, resulting in a reduction of WER by 6.1% and 3.6%, respectively. This underscores the robustness of the proposed EC method in addressing different ASR system outputs.

C. Experiments on Zero-shot Approach

This section will introduce zero-shot experiments and results using LLMs. Table IV presents the performance of ChatGPT (GPT-3.5 and GPT-4) for ASR error correction using either a Transducer-based ASR model or a Whisper model on the test_other data set. We compare zero-shot error correction results under different decoding constraints. GPT-3.5 with unconstrained decoding shows improvement, however, further analysis reveals an increase in deletion errors on the test sets. This is due to the fact that some sentences are truncated in the ChatGPT output, displaying only the initial few words instead of complete sentences. Constrained generation methods limit the output to the N-best list, effectively reducing deletions. The *closest* method, which identifies the closest hypothesis in the N-best list with the generated corrected one, outperforms methods that require ChatGPT to directly select the best option from the N-best list. Compared to GPT-3.5, GPT-4 shows improved performance on all test sets. The results indicate that for a powerful LLM like GPT-4, unconstrained decoding, giving the model more freedom, yields better results than constrained decoding approaches.

TABLE IV

RESULTS (% WER) FOR A CONFORMER-TRANSDUCER SYSTEM AND WHISPER WITH ZERO-SHOT ERROR CORRECTION USING GPTs.

System		Transducer			Whisper		
		LB	TED	Artie	LB	TED	Artie
Baseline		6.90	13.53	23.67	7.37	3.89	9.03
5-best Oracle		4.59	10.71	17.95	5.24	2.59	5.59
10-best T5		6.10	-	-	6.39	-	-
GPT-3.5	uncon	6.64	11.35	18.73	7.71	5.84	8.30
	constr	6.52	12.61	21.88	7.24	4.19	8.47
	closest	6.29	11.97	20.64	7.15	4.56	8.21
GPT-4	uncon	5.79	9.09	17.35	6.67	4.60	7.53
	constr	6.55	11.91	20.97	7.17	4.58	8.46
	closest	5.98	11.67	20.40	6.76	4.25	7.86

When comparing the zero-shot method with the fine-tuned N-best T5 model, we notice that GPT-3.5 matches the performance of the 10-best T5 for the Transducer ASR, while GPT-4 exceeds it across all three test sets. However, this success does not extend to outputs from the Whisper ASR, where LLMs in the zero-shot setting struggle more with error detection and correction. Using the closest match decoding approach, GPT-3.5 outperforms the baseline for the LB test set and Artie but does not reach the 10-best T5’s performance. Notably, even GPT-4 falls short of surpassing the baseline on the TED test set. The ineffectiveness of zero-shot methods on TED-LIUM3 decoded by Whisper will be discussed in Section VI-C). Specifically, GPT-4 achieves an average WERR of 25.2% on the three test sets for Transducer outputs, while only an average of 2.6% WERR is achieved on the three sets for Whisper outputs. This is likely due to the nature of the N-best lists, Section VI-C will discuss this in detail.

VI. DISCUSSION

A. Ablation on N-best Inputs

This section presents the ablation experiments with diverse N-best inputs for fine-tuning and zero-shot EC methods.

Fine-tuned EC approach: Table V presents results with baseline and N-best T5 models for the two ASR models on the LibriSpeech test_clean and test_other datasets. Training an EC model using the 1-best hypotheses from the Transducer ASR, as described in Section II, did not improve performance, highlighting the challenge of surpassing a strong baseline with limited input information. However, using 5-best and 10-best lists as model input, the T5 EC model can achieve relative performance gains of 6.0% and 7.7% on the test_other set, respectively. For the Whisper small.en model, the supervised error correction method showed performance gains even with the 1-best hypothesis, with improvements of 10.2% on the test_clean set and 4.6% on the test_other sets. Using the 5-best and 10-best lists yielded even better results, with the 10-best lists achieving a reduction of 17.6% and 13.3% on the test_clean and test_other sets, respectively. These findings indicate that while a larger N provides more diverse input and can improve error detection and correction, increasing N does not always guarantee proportional benefits.

Zero-shot EC approach: Table VI demonstrates the impact of varying N for the zero-shot error correction method, with

TABLE V

RESULTS FOR ASR BASELINE AND N-BEST T5 MODELS WITH DIFFERENT MODEL INPUTS ON LIBRISPEECH TEST_CLEAN AND TEST_OTHER SETS.

Model	Transducer		Whisper	
	clean	other	clean	other
Baseline	2.79	6.90	3.52	7.37
1-best T5	2.94	7.00	3.16	7.03
5-best T5	2.63	6.49	2.86	6.59
10-best T5	2.54	6.37	2.90	6.39

TABLE VI

ABLATION OF N-BEST LIST SIZES UTILIZING GPT-3.5 FOR ERROR CORRECTION ON CONFORMER-TRANSDUCER OUTPUTS ACROSS THREE TEST SETS WITH UNCONSTRAINED DECODING (%WER).

Method	Input	LB	TED	Artie
Baseline	-	6.90	13.53	23.67
GPT-3.5	1-best	8.25	11.95	21.19
	3-best	7.01	11.31	18.84
	5-best	6.64	11.35	18.73
	10-best	6.69	11.29	18.72

the details of the method introduced in Section II-C. Specifically, we tested the GPT-3.5 model on the Transducer outputs across three datasets: LibriSpeech test_other, TED-LIUM, and Artie, using the unconstrained decoding setup where the model generates sequences based on the input context. The model faces challenges in enhancing performance when using only the top one ASR hypothesis as input. Notably, while performance improved on TED and Artie datasets, it declined on LibriSpeech. Increasing the number of input contexts generally helps the model detect and correct errors more effectively. However, our findings indicate that increasing N does not consistently improve results; for example, performance with the 10-best context was comparable to that with the 5-best. This finding is consistent with the observation on the fine-tuning EC method.

B. Ablation on ASR Model Sizes

Previous experiments on Whisper are based on the small.en model, which yields good performance with relatively low decoding latency. In Table VII we test Whisper models with different sizes as the underlying ASR model and apply GPT-4 as the correction approach. The results indicate that increasing the ASR model size generally improves baseline ASR performance, but also makes the correction task more challenging. Specifically, error correction yields higher WERR with smaller Whisper models. Except for Whisper large-v2, our method achieves a WERR ranging from 7.5% to 17.6%, demonstrating the effectiveness of zero-shot error correction.

C. N-best Analysis

In Section VI-A, we observed that using an N-best list instead of the 1-best transcription significantly improves error correction performance. For the N-best T5 model, hypotheses are concatenated sequentially without explicitly encoding the ranking information. This raises an important question: Can

TABLE VII

ABLATION OF ZERO-SHOT ERROR CORRECTION RESULTS ON DIFFERENT SIZES OF WHISPER.

System	Oracle WER	Baseline				+GPT-4				WERR
		All	Sub	Del	Ins	All	Sub	Del	Ins	
base.en	6.75	9.50	7.1	1.0	1.4	7.91	5.6	1.0	1.3	17.6%
small.en	5.24	7.37	4.9	1.7	0.8	6.67	4.3	1.6	0.8	9.5%
medium.en	3.82	5.60	4.1	0.8	0.7	5.18	3.6	0.9	0.7	7.5%
large-v2	3.57	4.93	3.5	0.8	0.7	4.86	3.3	0.8	0.7	1.4%

TABLE VIII

ABLATION ANALYSIS WITH DISTURBED 10-BEST LIST FOR N-BEST T5 MODELS.

10-best	LibriSpeech		Other sets		
	clean	other	TED	Artie	MGB
Sorted	2.90	6.39	3.64	8.14	12.71
Randomized	3.31	6.82	3.74	8.50	13.01
Reversed	3.50	7.18	3.75	8.57	12.99

the correction model infer and utilize this ranking knowledge from the input to enhance its performance? To investigate this, we conducted experiments with N-best lists that were either randomly shuffled or sorted in reverse order of ASR scores. As shown in Table VIII, applying unconstrained decoding to the LibriSpeech test sets and N-best constrained decoding to other datasets, we found that randomizing the N-best list led to performance degradation, while reversing the order of input hypotheses resulted in the worst performance. This indicates that the ranking information is implicitly learned and crucial for the N-best T5 model to perform well. However, when applying similar randomization or reversal strategies to the GPT-3.5 and GPT-4 models in zero-shot experiments, there was no significant difference in performance. This suggests that, unlike the N-best T5 model, the GPTs might not rely on or benefit from the ranking information in the same way.

Experiments in Section V-C reveal that our proposed methods are less effective on Whisper outputs in some cases. To examine this, we calculate *Uniq* and *Cross WER* metrics in Table IX. When calculating statistics, we remove punctuation and special symbols from the ASR hypotheses, leaving only English characters and numbers to focus on the meaningful content. The *Uniq* metric represents the average number of unique hypotheses within an N-best list in the test set. For Transducer outputs, this number is close to 5, matching the size of the given N-best list. However, Whisper outputs show more repeated entries. This occurs because Whisper learns to generate sentences with inverse text normalisation (ITN) to enhance readability, i.e. adding capitalisation, including punctuation, and removing disfluencies. As a result, multiple hypotheses in an N-best list often differ only in format rather than content. This limits the diversity of the N-best list which is crucial for our proposed methods to work well.

Another notable observation is that for Whisper, even when the N-best list contains diverse hypotheses, the differences often come from the omission or insertion of irrelevant words. This is demonstrated by the Cross WER metric in Table IX. In this evaluation, we retain all unique hypotheses in an N-best

TABLE IX
STATISTICS OF THE ASR 5-BEST LISTS GENERATED BY THE CONFORMER-TRANSDUCER AND THE WHISPER MODEL ON LIBRISPEECH (LB), TED-LIUM3 (TED) AND ARTIE BIAS TEST SETS.

Data	Model	Uniq	% Cross			
			All	Sub	Del	Ins
LB	Transducer	4.9	9.1	7.1	1.0	1.0
	Whisper	3.0	12.9	7.5	2.7	2.7
TED	Transducer	5.0	7.4	5.4	1.0	1.0
	Whisper	2.6	9.9	3.9	3.0	3.0
Artie	Transducer	4.8	19.9	15.3	2.3	2.3
	Whisper	2.9	21.1	14.5	3.3	3.3

list and then calculate the WER between each pair of hypotheses, summing the results for the entire set. This metric helps measure the difference between hypotheses within the same N-best list. The results indicate that Whisper has significantly higher deletion and insertion rates on Cross WER compared to the Transducer model, particularly on TED-LIUM3. This suggests that Whisper may struggle to consistently transcribe utterances accurately across all N-best hypotheses, resulting in sentences of varying lengths. ChatGPT tends to select the more coherent hypotheses in the zero-shot setting, leading to a higher rate of deletion errors in the output.

In Table X we demonstrate the outputs from different models on a specific example. Here, the unconstrained decoding is employed when 5-best lists are used as the model input for error correction. Compared to the desired reference, we highlight the errors in the generated ASR hypotheses and the error correction outputs from LLMs. The word *ligatures*, a medical term related to surgery, is wrongly transcribed in the Top-1 ASR hypothesis. However, it appears in the correct format in the second-best hypothesis. With the T5 model, ASR errors are not recovered in the output. Meanwhile, as GPT models are pre-trained on more data, they present a better understanding of general world knowledge, leading to fewer errors utilizing the given contextual information.

D. Multi-Model N-best Lists

In previous experiments, we demonstrated that LLMs can make use of N-best lists generated by a single ASR model to perform error correction and enhance ASR performance. In this section, we extend this approach by combining the N-best decoding hypotheses from different ASR systems with LLMs. We experiment with two scenarios: (a) combining outputs from ASR models with different architectures and (b) combining outputs from ASR models trained on different datasets. ASR systems with different architectures exhibit unique strengths and weaknesses. For instance, a LAS model is good at utilizing global context information from the given input but tends to be less robust. An RNN-T model alleviates the problem of repeating and skipping word chunks compared to a LAS model although it shows worse performance in general. By combining outputs, a more robust ASR system that takes advantage of different components can be built. Additionally, for ASR models trained on diverse datasets, combining the

outputs acts as a form of model ensembling, thus is expected to improve the system performance.

We also draw on the Recognizer Output Voting Error Reduction (ROVER) technique, which employs a majority voting approach to combine the recognition results of several ASR systems into a single recognition hypothesis [49]. ROVER converts multiple ASR outputs into Word Transition Networks (WTNs), aligns and combines these WTNs using edit distance, and then uses weighted voting to determine the final hypothesis. Since ROVER is a simple, training-free technique for integrating information from different sentences, we use it as a baseline in the experiments.

In Table XI, we list the ASR error correction results using the N-best list generated by a single system, denoted as $T_1T_2T_3T_4T_5$ (shortened as T) for Transducer outputs and $E_1E_2E_3E_4E_5$ (shortened as E) for Whisper small.en outputs. T_i and E_i refer to the i -best hypothesis generated by the Transducer and the Whisper model respectively. For the output combination experiments, we take the 5-best lists generated by both the Transducer and the Whisper model for each test utterance. There are multiple ways to combine the two N-best lists to form a new 5-best list. In our preliminary experiments, we altered different ways of combining the N-best hypotheses and different sentence orders to find the best combination. We use ROVER to determine the performance of these different inputs, achieving the best performance with an input of $E_1E_2T_1T_2T_3$, denoted as $Comb1$ in Table XI, yielding a WER of 5.95%. Experiments on GPT models show that using outputs from diverse systems rather than a single system leads to performance boosts. LLMs could utilize information from both model outputs to generate a more robust answer. The best WER performance is 4.72%, which is 32% and 36% lower than the Transducer ASR and Whisper baselines, respectively.

Additionally, we combine the ASR N-best lists generated by two different versions of Whisper models – small and small.en, denoted as S for N-best list $S_1S_2S_3S_4S_5$ from Whisper small model and E for N-best list $E_1E_2E_3E_4E_5$ from Whisper small.en model, respectively. Both models are the same size but are trained on different training data. The small.en model was pre-trained on English-only weakly supervised data, while the small one was pre-trained on a larger, multilingual dataset. Experiments on ROVER show that the combination of $S_1S_2E_1E_2E_3$ ($Comb2$) works the best, leading to a WER of 6.82% on the test set. The zero-shot error correction results indicate that LLMs serve as an effective method for model ensembling. With *uncon* decoding using GPT-4, WERRs of 23% and 21% over ASR baselines can be seen on the test set. This showcases that LLMs can effectively improve ASR accuracy.

E. Potential Data Contamination

Previous results highlight the effectiveness of zero-shot error correction using LLMs, attributed to their robust language understanding capabilities. However, a key concern with this approach is the possibility that text from ASR test sets may have been included in the LLM pre-training data, potentially

TABLE X
CASE ANALYSIS FOR UNCONDITIONAL ERROR CORRECTION RESULTS ON CONFORMER-TRANSDUCER OUTPUTS.

Ref	the gut and the gullet being cut across between these ligatures the stomach may be removed entire without spilling its contents
Hyp-1	the gut and the gullet being cut across between these ligatches the stomach may be removed entire without spinning its contents
Hyp-2	the gut and the gullet being cut across between these ligatures the stomach may be removed entire without spinning its contents
Hyp-3	the gut and the gullet being cut across between these ligages the stomach may be removed entire without spinning its contents
Hyp-4	the gut and the gullet being cut across between these ligatches the stomach may be removed entire without spinning as contents
Hyp-5	the gut and the gullet being cut across between these ligatches the stomach may be removed entire without spinning his contents
5-best T5	the gut and the gullet being cut across between these ligatches the stomach may be removed entire without spinning its contents
GPT-3.5	The gut and the gullet being cut across between these ligatures the stomach may be removed entire without spinning its contents.
GPT-4	The gut and the gullet being cut across between these ligatures the stomach may be removed entire without spilling its contents.

TABLE XI
SYSTEM COMBINATION RESULTS ON TEST_OTHER USING 5-BEST LISTS FROM TRANSDUCER (*T*), WHISPER SMALL. EN (*E*) AND SMALL (*S*) MODELS.

N-best Input		T	E	S	Comb1	Comb2
Baseline		6.90	7.37	7.20	5.95	6.82
5-best Oracle		4.59	5.24	5.21	3.35	4.40
GPT-3.5	uncon	6.64	7.71	7.46	6.78	7.08
	closest	6.29	7.15	7.06	6.01	6.46
GPT-4	uncon	5.79	6.67	6.49	4.72	5.70
	closest	5.98	6.76	6.51	5.00	5.80

leading to biased evaluations in error correction tasks. In this section, we explore the potential issue of data leakage from ASR test sets during LLM pre-training.

Following the practice in [39], we randomly select 100 utterances from each test set for evaluation. In addition to the three public test sets, we also applied our proposed method to two internal ASR datasets: MGB-3 [50] and Linguaskill [51] that are less likely to be contaminated. These internal datasets are less susceptible to contamination due to their unique characteristics and restricted access. MGB-3 is a dataset specifically designed for the multi-genre broadcast challenge, containing broadcast media recordings that are carefully curated and controlled. Linguaskill, on the other hand, consists of educational and skill-based assessments that are not publicly available, ensuring a low risk of data contamination. Results on these datasets are intended to be contrasted with the results on the public datasets.

For the designed data contamination quiz, the LLM is asked to identify which sentence is from the ASR test set, choosing between an actual ASR test reference and a rewritten sentence. A lower percentage of correct selections by the LLM indicates a less severe data contamination. The results in Table XII suggest that data contamination is not a significant issue for GPT-3.5. Although GPT-4 shows some level of data contamination on LibriSpeech and TED-LIUM3, this does not invalidate our method. The potential slight contamination observed with GPT-4 highlights an area for future improvement and caution but does not undermine the robustness and effectiveness of our proposed approach. The method shows consistent performance improvements across different datasets and ASR systems, indicating its generalizability and robustness despite the slight contamination in some instances.

TABLE XII
RESULTS OF THE DATA CONTAMINATION QUIZ. A LOWER PERCENTAGE IMPLIES LESS CONTAMINATION IN THE LLM PRE-TRAINING.

Datasets	GPT-3.5	GPT-4
LibriSpeech (test_other)	0.07	0.33
TED-LIUM 3 (test)	0.05	0.22
Artie Bias (test)	0.14	0.15
MGB-3 (test)	0.04	0.02
Linguaskill (ling_test_general)	0.03	0.09

VII. CONCLUSION

In this work, we proposed and thoroughly investigated two advanced error correction methods to enhance ASR accuracy: supervised EC with pre-trained language models and zero-shot EC with LLMs. Various decoding strategies were explored for both supervised and zero-shot EC methods, including unconstrained decoding, N-best constrained decoding, and closest mapping decoding, each offering unique advantages in different scenarios. Our experiments demonstrated the robustness and generalization capabilities of the proposed methods across multiple dimensions. First, we tested our models trained on outputs from a specific ASR on outputs from different ASR systems, showcasing their adaptability. Second, we evaluated the models on datasets from diverse domains and applied an EC model trained on one dataset to other datasets, proving the versatility of the method. We also extended the approach to incorporate N-best lists from multiple ASR systems, demonstrating the model can serve as effective model ensembling. Another crucial aspect of our study was addressing the potential data contamination issue, particularly in the use of LLMs for ASR error correction. Our systematic evaluation, using a combination of public and proprietary datasets to ensure comprehensive coverage, revealed minimal contamination in GPT-3.5 but identified some level of contamination in GPT-4, especially on certain datasets. These findings underline the importance of vigilance in data handling and provide valuable guidelines for future research.

ACKNOWLEDGMENTS

This paper reports on research supported by EPSRC Project EP/V006223/1 (Multimodal Video Search by Examples) and Cambridge University Press & Assessment, a department of The Chancellor, Masters, and Scholars of the University of Cambridge.

REFERENCES

- [1] C. M. Rebman Jr, M. W. Aiken, and C. G. Cegielski, "Speech recognition in the human-computer interface," *Information & Management*, vol. 40, no. 6, pp. 509–519, 2003.
- [2] W. Chan, N. Jaitly, Q. Le, and O. Vinyals, "Listen, Attend and Spell: A neural network for large vocabulary conversational speech recognition," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2016, pp. 4960–4964.
- [3] A. Graves and N. Jaitly, "Towards end-to-end speech recognition with recurrent neural networks," in *Proc. International Conference on Machine Learning*. PMLR, 2014, pp. 1764–1772.
- [4] D. Amodei, S. Ananthanarayanan, R. Anubhai, J. Bai, E. Battenberg, C. Case, J. Casper, B. Catanzaro, Q. Cheng, G. Chen *et al.*, "Deep Speech 2: End-to-end speech recognition in English and Mandarin," in *International conference on machine learning*. PMLR, 2016, pp. 173–182.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [6] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *International Conference on Machine Learning*. PMLR, 2023, pp. 28492–28518.
- [7] Y. Zhang, W. Han, J. Qin, Y. Wang, A. Bapna, Z. Chen, N. Chen, B. Li, V. Axelrod, G. Wang *et al.*, "Google USM: Scaling Automatic Speech Recognition Beyond 100 Languages," *arXiv preprint arXiv:2303.01037*, 2023.
- [8] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur, "Recurrent neural network based language model," in *Proc. Interspeech 2010*, 2010, pp. 1045–1048.
- [9] Z. Meng, S. Parthasarathy, E. Sun, Y. Gaur, N. Kanda, L. Lu, X. Chen, R. Zhao, J. Li, and Y. Gong, "Internal language model estimation for domain-adaptive end-to-end speech recognition," in *2021 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2021, pp. 243–250.
- [10] E. McDermott, H. Sak, and E. Variani, "A density ratio approach to language model fusion in end-to-end automatic speech recognition," in *2019 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2019, pp. 434–441.
- [11] Y. Liu, R. Ma, H. Xu, Y. He, Z. Ma, and W. Zhang, "Internal Language Model Estimation Through Explicit Context Vector Learning for Attention-based Encoder-decoder ASR," in *Proc. Interspeech 2022*, 2022, pp. 1666–1670.
- [12] M. ZeinEldien, A. Glushko, W. Michel, A. Zeyer, R. Schlüter, and H. Ney, "Investigating Methods to Improve Language Model Integration for Attention-Based Encoder-Decoder ASR Models," in *Proc. Interspeech 2021*, 2021, pp. 2856–2860.
- [13] R. Errattahi, A. El Hannani, and H. Ouahmane, "Automatic speech recognition errors detection and correction: A review," *Procedia Computer Science*, vol. 128, pp. 32–37, 2018.
- [14] O. Hrinchuk, M. Popova, and B. Ginsburg, "Correction of automatic speech recognition with transformer sequence-to-sequence model," in *Proc. 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 7074–7078.
- [15] Y. Zhao, X. Yang, J. Wang, Y. Gao, C. Yan, and Y. Zhou, "BART Based Semantic Correction for Mandarin Automatic Speech Recognition System," in *Proc. Interspeech 2021*, 2021, pp. 2017–2021.
- [16] H. Cucu, A. Buzo, L. Besacier, and C. Burileanu, "Statistical error correction methods for domain-specific ASR systems," in *Statistical Language and Speech Processing: First International Conference, SLSP 2013, Tarragona, Spain, July 29-31, 2013. Proceedings 1*. Springer, 2013, pp. 83–92.
- [17] Y. Ren, Y. Ruan, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "FastSpeech: Fast, robust and controllable text to speech," *Advances in neural information processing systems*, vol. 32, 2019.
- [18] J. Guo, T. N. Sainath, and R. J. Weiss, "A spelling correction model for end-to-end speech recognition," in *Proc. 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2019, pp. 5651–5655.
- [19] A. Mani, S. Palaskar, N. V. Meripo, S. Konam, and F. Metze, "ASR error correction and domain adaptation using machine translation," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 6344–6348.
- [20] K. Shen, Y. Leng, X. Tan, S. Tang, Y. Zhang, W. Liu, and E. Lin, "Mask the Correct Tokens: An Embarrassingly Simple Approach for Error Correction," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2022, pp. 10367–10380.
- [21] R. Ma, M. J. F. Gales, K. M. Knill, and M. Qian, "N-best T5: Robust ASR Error Correction using Multiple Input Hypotheses and Constrained Decoding Space," in *Proc. INTERSPEECH*, 2023, pp. 3267–3271.
- [22] R. Ma, M. Qian, M. J. F. Gales, and K. M. Knill, "Adapting an Unadaptable ASR System," in *Proc. INTERSPEECH 2023*, 2023, pp. 989–993.
- [23] H. Wu, W. Wang, Y. Wan, W. Jiao, and M. Lyu, "ChatGPT or Grammarly? evaluating ChatGPT on grammatical error correction benchmark," *arXiv preprint arXiv:2303.13648*, 2023.
- [24] T. Fang, S. Yang, K. Lan, D. F. Wong, J. Hu, L. S. Chao, and Y. Zhang, "Is ChatGPT a highly fluent grammatical error correction system? a comprehensive evaluation," *arXiv preprint arXiv:2304.01746*, 2023.
- [25] R. Ma, M. Qian, P. Manakul, M. Gales, and K. Knill, "Can generative large language models perform asr error correction?" *arXiv preprint arXiv:2307.04172*, 2023.
- [26] K. Everson, Y. Gu, H. Yang, P. G. Shivakumar, G.-T. Lin, J. Kolehmainen, I. Bulyko, A. Gandhe, S. Ghosh, W. Hamza *et al.*, "Towards ASR robust spoken language understanding through in-context learning with word confusion networks," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 12856–12860.
- [27] C. Chen, Y. Hu, C.-H. H. Yang, S. M. Siniscalchi, P.-Y. Chen, and E.-S. Chng, "Hyporadise: An open baseline for generative speech recognition with large language models," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 2023, pp. 31665–31688.
- [28] Y. Hu, C. Chen, C. Qin, Q. Zhu, E. S. Chng, and R. Li, "Listen again and choose the right answer: A new paradigm for automatic speech recognition with large language models," *arXiv preprint arXiv:2405.10025*, 2024.
- [29] S. Li, C. Chen, C. Y. Kwok, C. Chu, E. S. Chng, and H. Kawai, "Investigating asr error correction with large language model and multilingual 1-best hypotheses," in *Interspeech 2024*, 2024, pp. 1315–1319.
- [30] M. Sundermeyer, R. Schlüter, and H. Ney, "Lstm neural networks for language modeling," in *Interspeech*, vol. 2012, 2012, pp. 194–197.
- [31] L. Zhu, W. Liu, L. Liu, and E. Lin, "Improving ASR error correction using n-best hypotheses," in *2021 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2021, pp. 83–89.
- [32] Y. Leng, X. Tan, R. Wang, L. Zhu, J. Xu, W. Liu, L. Liu, X.-Y. Li, T. Qin, E. Lin *et al.*, "FastCorrect 2: Fast Error Correction on Multiple Candidates for Automatic Speech Recognition," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021, pp. 4328–4337.
- [33] X. Liu, M. Li, L. Chen, P. Wanigasekara, W. Ruan, H. Khan, W. Hamza, and C. Su, "ASR N-best fusion nets," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 7618–7622.
- [34] K. Ganesan, P. Bamdev, B. Jaivarsan, A. Venugopal, and A. Tushar, "N-Best ASR Transformer: Enhancing SLU Performance using Multiple ASR Hypotheses," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, 2021, pp. 93–98.
- [35] S. Dathathri, A. Madotto, J. Lan, J. Hung, E. Frank, P. Molino, J. Yosinski, and R. Liu, "Plug and Play Language Models: A Simple Approach to Controlled Text Generation," in *Proc. International Conference on Learning Representations*, 2020.
- [36] M. Auli, M. Galley, C. Quirk, and G. Zweig, "Joint language and translation modeling with recurrent neural networks," in *Proc. EMNLP*, 2013.
- [37] O. Sainz, J. Campos, I. García-Ferrero, J. Etxaniz, O. L. de Lacalle, and E. Agirre, "NLP Evaluation in trouble: On the Need to Measure LLM Data Contamination for each Benchmark," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 10776–10787.
- [38] C. Li and J. Flanagan, "Task contamination: Language models may not be few-shot anymore," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 18471–18480.
- [39] S. Golchin and M. Surdeanu, "Data contamination quiz: A tool to detect and estimate contamination in large language models," *arXiv preprint arXiv:2311.06233*, 2023.
- [40] A. Liusie, P. Manakul, and M. Gales, "Mitigating word bias in zero-shot prompt-based classifiers," in *Findings of the Association for Computational Linguistics: IJCNLP-AACL 2023 (Findings)*, 2023, pp. 327–335.
- [41] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: an ASR corpus based on public domain audio books," in *2015 IEEE*

- international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2015, pp. 5206–5210.
- [42] F. Hernandez, V. Nguyen, S. Ghannay, N. Tomashenko, and Y. Esteve, “TED-LIUM 3: Twice as much data and corpus repartition for experiments on speaker adaptation,” in *Speech and Computer: 20th International Conference, SPECOM 2018, Leipzig, Germany, September 18–22, 2018, Proceedings 20*. Springer, 2018, pp. 198–208.
 - [43] J. Meyer, L. Rauchenstein, J. D. Eisenberg, and N. Howell, “Artie bias corpus: An open dataset for detecting demographic bias in speech applications,” in *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 2020, pp. 6462–6468.
 - [44] R. Ardila, M. Branson, K. Davis, M. Kohler, J. Meyer, M. Henretty, R. Morais, L. Saunders, F. Tyers, and G. Weber, “Common Voice: A Massively-Multilingual Speech Corpus,” in *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 2020, pp. 4218–4222.
 - [45] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu *et al.*, “Conformer: Convolution-augmented Transformer for Speech Recognition,” in *Proc. Interspeech 2020*, 2020, pp. 5036–5040.
 - [46] S. Watanabe, T. Hori, S. Karita, T. Hayashi, J. Nishitoba, Y. Unno, N.-E. Y. Soplin, J. Heymann, M. Wiesner, N. Chen *et al.*, “ESPnet: End-to-End Speech Processing Toolkit,” in *Proc. Interspeech 2018*, 2018, pp. 2207–2211.
 - [47] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition,” in *Proc. Interspeech 2019*, 2019, pp. 2613–2617.
 - [48] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *International Conference on Learning Representations*, 2019.
 - [49] J. G. Fiscus, “A post-processing system to yield reduced word error rates: Recognizer output voting error reduction (ROVER),” in *IEEE Workshop on Automatic Speech Recognition and Understanding Proceedings*. IEEE, 1997, pp. 347–354.
 - [50] P. Bell, M. J. Gales, T. Hain, J. Kilgour, P. Lanchantin, X. Liu, A. McParland, S. Renals, O. Saz, M. Wester *et al.*, “The MGB challenge: Evaluating multi-genre broadcast media recognition,” in *2015 IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU)*. IEEE, 2015, pp. 687–693.
 - [51] J. Xu, M. Brenchley, E. Jones, A. Pinnington, T. Benjamin, K. Knill, G. Seal-Coon, M. Robinson, and A. Geranpayeh, “Linguaskill building a validity argument for the speaking test,” *Linguaskill Research Reports, UCLES, Tech. Rep*, 2020.